

```

#include <stdio.h>
#include <omp.h>

int main() {
    int num_threads;

    printf("Enter number of threads: ");
    scanf("%d", &num_threads);

    omp_set_num_threads(num_threads);

    printf("\n--- Sequential Execution ---\n");
    for (int i = 0; i < num_threads; i++) {
        printf("Hello, World from thread %d out of %d (Sequential)\n", i,
num_threads);
    }

    printf("\n--- Parallel Execution ---\n");

    #pragma omp parallel
    {
        int tid = omp_get_thread_num();
        int total = omp_get_num_threads();
        printf("Hello, World from thread %d out of %d (Parallel)\n", tid,
total);
    }

    return 0;
}

```

Vector Scalar Addition

```

#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

void vector_scalar_addition(float *A, float *C, float scalar, int N, int
num_threads) {
    #pragma omp parallel for num_threads(num_threads)
    for (int i = 0; i < N; i++) {
        C[i] = A[i] + scalar;
    }
}

```

```

int main() {
    int N_values[] = {1000000, 10000000, 50000000};
    int thread_counts[] = {1, 2, 4, 8};
    float scalar = 3.5;

    FILE *fp = fopen("PS_1.csv", "w");
    if (fp == NULL) {
        perror("Error opening file");
        return 1;
    }

    fprintf(fp, "DataSize,Threads,Time\n");

    printf("%-12s %-7s %s\n", "DataSize", "Threads", "Time(s)");
    printf("-----\n");

    for (int ni = 0; ni < sizeof(N_values)/sizeof(N_values[0]); ni++) {
        int N = N_values[ni];

        float *A = (float *)malloc(N * sizeof(float));
        float *C = (float *)malloc(N * sizeof(float));

        for (int i = 0; i < N; i++) {
            A[i] = i * 0.5;
        }

        for (int t = 0; t < sizeof(thread_counts)/sizeof(thread_counts[0]);
t++) {
            int num_threads = thread_counts[t];
            double start = omp_get_wtime();

            vector_scalar_addition(A, C, scalar, N, num_threads);

            double end = omp_get_wtime();
            double elapsed = end - start;

            printf("%-12d %-7d %.6f\n", N, num_threads, elapsed);

            fprintf(fp, "%d,%d,%.6f\n", N, num_threads, elapsed);
        }

        free(A);
        free(C);
    }

    fclose(fp);
    return 0;
}

```

Calculate value of π

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

double calculate_pi(int num_steps, int num_threads) {
    double step = 1.0 / (double)num_steps;
    double sum = 0.0;

    #pragma omp parallel for num_threads(num_threads) reduction(+:sum)
    for (int i = 0; i < num_steps; i++) {
        double x = (i + 0.5) * step;
        sum += 1.0 / (1.0 + x * x);
    }

    return 4.0 * step * sum;
}

int main() {
    int steps[] = {1000000, 10000000, 50000000};
    int threads[] = {1, 2, 4, 8};

    FILE *fp = fopen("PS_2.csv", "w");
    if (fp == NULL) {
        perror("Error opening file");
        return 1;
    }

    fprintf(fp, "Steps,Threads,Time(s),ApproxPi\n");

    printf("%-12s %-8s %-10s %-20s\n", "Steps", "Threads", "Time(s)",
"Approx. Pi");

    for (int s = 0; s < sizeof(steps)/sizeof(steps[0]); s++) {
        int num_steps = steps[s];

        for (int t = 0; t < sizeof(threads)/sizeof(threads[0]); t++) {
            int num_threads = threads[t];
            double start = omp_get_wtime();

            double pi = calculate_pi(num_steps, num_threads);

            double end = omp_get_wtime();
            printf("%-12d %-8d %-10.6f %-20.15f\n", num_steps, num_threads,
end - start, pi);
        }
    }
}
```

```

        fprintf(fp, "%d,%d,%.6f,%.15f\n", num_steps, num_threads, end -
start, pi);
    }
}

fclose(fp);
printf("Results saved to PS_2.csv\n");

return 0;
}

```

Minimum Scalar Product

```

#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
#include <time.h>

int compare_asc(const void *a, const void *b) {
    return (*(int *)a - *(int *)b);
}

int compare_desc(const void *a, const void *b) {
    return (*(int *)b - *(int *)a);
}

void generate_random_vector(int *vec, int n) {
    for (int i = 0; i < n; i++) {
        vec[i] = rand() % 100;
    }
}

long long min_scalar_product(int *A, int *B, int n, int num_threads) {
    qsort(A, n, sizeof(int), compare_asc);
    qsort(B, n, sizeof(int), compare_desc);

    long long result = 0;

    #pragma omp parallel for reduction(+:result) num_threads(num_threads)
    for (int i = 0; i < n; i++) {
        result += (long long)A[i] * B[i];
    }

    return result;
}

```

```

int main() {
    int sizes[] = {1000000, 10000000, 50000000};
    int threads[] = {1, 2, 4, 8};
    FILE *fp = fopen("PS_1.csv", "w");

    if (!fp) {
        perror("Error opening file");
        return 1;
    }

    srand(time(NULL));

    fprintf(fp, "Vector_Size,Threads,Time(s),Min_Scalar_Product\n");
    printf("%-12s %-8s %-10s %-20s\n", "Size", "Threads", "Time(s)", "Min
Scalar Product");

    for (int s = 0; s < sizeof(sizes)/sizeof(sizes[0]); s++) {
        int n = sizes[s];

        for (int t = 0; t < sizeof(threads)/sizeof(threads[0]); t++) {
            int thread_count = threads[t];

            int *A = (int *)malloc(n * sizeof(int));
            int *B = (int *)malloc(n * sizeof(int));

            generate_random_vector(A, n);
            generate_random_vector(B, n);

            double start = omp_get_wtime();
            long long result = min_scalar_product(A, B, n, thread_count);
            double end = omp_get_wtime();
            double elapsed = end - start;

            printf("%-12d %-8d %-10.6f %-20lld\n", n, thread_count, elapsed,
result);
            fprintf(fp, "%d,%d,%.6f,%lld\n", n, thread_count, elapsed,
result);

            free(A);
            free(B);
        }
    }

    fclose(fp);
    return 0;
}

```

2D Matrix Addition

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
#include <time.h>

void generate_random_matrix(int **matrix, int size) {
    for (int i = 0; i < size; i++)
        for (int j = 0; j < size; j++)
            matrix[i][j] = rand() % 100;
}

void matrix_addition(int **A, int **B, int **C, int size, int threads) {
    #pragma omp parallel for collapse(2) num_threads(threads)
    for (int i = 0; i < size; i++)
        for (int j = 0; j < size; j++)
            C[i][j] = A[i][j] + B[i][j];
}

int **allocate_matrix(int size) {
    int **matrix = (int **)malloc(size * sizeof(int *));
    for (int i = 0; i < size; i++)
        matrix[i] = (int *)malloc(size * sizeof(int));
    return matrix;
}

void free_matrix(int **matrix, int size) {
    for (int i = 0; i < size; i++)
        free(matrix[i]);
    free(matrix);
}

int main() {
    int sizes[] = {250, 500, 750, 1000, 2000};
    int thread_counts[] = {1, 2, 4, 8};

    FILE *fp = fopen("PS_2.csv", "w");
    if (!fp) {
        perror("Error opening file");
        return 1;
    }

    srand(time(NULL));
    fprintf(fp, "Matrix_Size,Threads,Time(s)\n");
    printf("%-12s %-8s %-10s\n", "Size", "Threads", "Time(s)");

    for (int s = 0; s < sizeof(sizes) / sizeof(sizes[0]); s++) {
        int size = sizes[s];
```

```

        for (int t = 0; t < sizeof(thread_counts) /
sizeof(thread_counts[0]); t++) {
            int threads = thread_counts[t];

            int **A = allocate_matrix(size);
            int **B = allocate_matrix(size);
            int **C = allocate_matrix(size);

            generate_random_matrix(A, size);
            generate_random_matrix(B, size);

            double start = omp_get_wtime();
            matrix_addition(A, B, C, size, threads);
            double end = omp_get_wtime();
            double elapsed = end - start;

            printf("%-12d %-8d %-10.6f\n", size, threads, elapsed);
            fprintf(fp, "%d,%d,%.6f\n", size, threads, elapsed);

            free_matrix(A, size);
            free_matrix(B, size);
            free_matrix(C, size);
        }
    }

    fclose(fp);
    return 0;
}

```

Vector Addition (with Static, Dynamic Scheduling)

```

#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
#include <time.h>

#define VECTOR_SIZE 1000000
#define SCALAR 10

void generate_vector(int *vec, int size) {
    for (int i = 0; i < size; i++)
        vec[i] = rand() % 100;
}

void vector_scalar_add_static(int *vec, int *result, int size, int chunk,

```

```

int threads) {
    #pragma omp parallel for schedule(static, chunk) num_threads(threads)
    for (int i = 0; i < size; i++) {
        result[i] = vec[i] + SCALAR;
    }
}

void vector_scalar_add_dynamic(int *vec, int *result, int size, int chunk,
int threads) {
    #pragma omp parallel for schedule(dynamic, chunk) num_threads(threads)
    for (int i = 0; i < size; i++) {
        result[i] = vec[i] + SCALAR;
    }
}

void nowait_demo(int *vec, int *res1, int *res2, int size, int threads) {
    #pragma omp parallel num_threads(threads)
    {
        #pragma omp for nowait
        for (int i = 0; i < size / 2; i++)
            res1[i] = vec[i] + 1;

        #pragma omp for nowait
        for (int i = size / 2; i < size; i++)
            res2[i] = vec[i] + 2;
    }
}

int main() {
    int chunk_sizes[] = {1, 5, 10, 20, 50};
    int thread_counts[] = {1, 2, 4, 8};

    int *vec = (int *)malloc(VECTOR_SIZE * sizeof(int));
    int *res_static = (int *)malloc(VECTOR_SIZE * sizeof(int));
    int *res_dynamic = (int *)malloc(VECTOR_SIZE * sizeof(int));
    int *res1 = (int *)calloc(VECTOR_SIZE, sizeof(int));
    int *res2 = (int *)calloc(VECTOR_SIZE, sizeof(int));

    srand(time(NULL));
    generate_vector(vec, VECTOR_SIZE);

    FILE *fs = fopen("PS_3_Static.csv", "w");
    FILE *fd = fopen("PS_3_Dynamic.csv", "w");
    fprintf(fs, "Chunk_Size,Threads,Time(s)\n");
    fprintf(fd, "Chunk_Size,Threads,Time(s)\n");

    printf("STATIC SCHEDULING:\n");
    printf("%-12s %-8s %-10s\n", "Chunk", "Threads", "Time(s)");
    for (int c = 0; c < sizeof(chunk_sizes)/sizeof(chunk_sizes[0]); c++) {
        for (int t = 0; t < sizeof(thread_counts)/sizeof(thread_counts[0]);

```



```

t++) {
    double start = omp_get_wtime();
    vector_scalar_add_static(vec, res_static, VECTOR_SIZE,
chunk_sizes[c], thread_counts[t]);
    double end = omp_get_wtime();
    double elapsed = end - start;

    printf("%-12d %-8d %-10.6f\n", chunk_sizes[c], thread_counts[t],
elapsed);
    fprintf(fs, "%d,%d,%.6f\n", chunk_sizes[c], thread_counts[t],
elapsed);
}
}

printf("\nDYNAMIC SCHEDULING:\n");
printf("%-12s %-8s %-10s\n", "Chunk", "Threads", "Time(s)");
for (int c = 0; c < sizeof(chunk_sizes)/sizeof(chunk_sizes[0]); c++) {
    for (int t = 0; t < sizeof(thread_counts)/sizeof(thread_counts[0]);
t++) {
        double start = omp_get_wtime();
        vector_scalar_add_dynamic(vec, res_dynamic, VECTOR_SIZE,
chunk_sizes[c], thread_counts[t]);
        double end = omp_get_wtime();
        double elapsed = end - start;

        printf("%-12d %-8d %-10.6f\n", chunk_sizes[c], thread_counts[t],
elapsed);
        fprintf(fd, "%d,%d,%.6f\n", chunk_sizes[c], thread_counts[t],
elapsed);
    }
}

fclose(fs);
fclose(fd);

printf("\nNOWAIT CLAUSE DEMO (partial result shown):\n");
nowait_demo(vec, res1, res2, VECTOR_SIZE, 4);
for (int i = 0; i < 10; i++)
    printf("res1[%d] = %d, res2[%d] = %d\n", i, res1[i], i, res2[i]);

free(vec);
free(res_static);
free(res_dynamic);
free(res1);
free(res2);

return 0;
}

```

Fibonacci Sequence

```
#include <stdio.h>
#include <omp.h>

long fib(int n) {
    if (n < 2) return n;

    long x, y;

    #pragma omp task shared(x)
    x = fib(n - 1);

    #pragma omp task shared(y)
    y = fib(n - 2);

    #pragma omp taskwait
    return x + y;
}

int main() {
    int n = 10;
    long result;

    double start = omp_get_wtime();

    #pragma omp parallel
    {
        #pragma omp single
        result = fib(n);
    }

    double end = omp_get_wtime();

    printf("Fibonacci(%d) = %ld\n", n, result);
    printf("Time: %f seconds\n", end - start);

    return 0;
}
```

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

long fib_iter(int n) {
    if (n < 2) return n;
```

```

    long a = 0, b = 1, c;
    for (int i = 2; i <= n; i++) {
        c = a + b;
        a = b;
        b = c;
    }
    return b;
}

int main() {
    int nums[] = {10, 20, 25, 30};
    int size = 4;
    long *results = malloc(size * sizeof(long));
    if (results == NULL) {
        fprintf(stderr, "Memory allocation failed\n");
        return 1;
    }

    double start = omp_get_wtime();

    #pragma omp parallel for schedule(dynamic) shared(results)
    for (int i = 0; i < size; i++) {
        results[i] = fib_iter(nums[i]);
        #pragma omp critical
        printf("Thread %d computed fib(%d) = %ld\n",
            omp_get_thread_num(), nums[i], results[i]);
    }

    double end = omp_get_wtime();
    printf("\nTotal time: %f seconds\n", end - start);
    free(results);
    return 0;
}

```

Producer Consumer

```

#include <stdio.h>
#include <omp.h>
#include <windows.h>

#define BUFFER_SIZE 5
#define NUM_ITEMS 20

int buffer[BUFFER_SIZE];
int count = 0;
int in = 0;

```

```

int out = 0;

void produce_item(int item) {
    Sleep(100);
    printf("Produced: %d\n", item);
}

void consume_item(int item) {
    Sleep(150);
    printf("Consumed: %d\n", item);
}

int main() {
    int i;

    omp_set_num_threads(2);

    #pragma omp parallel shared(buffer, count, in, out)
    {
        int thread_id = omp_get_thread_num();

        if (thread_id == 0) {
            for (i = 1; i <= NUM_ITEMS; i++) {
                int produced = 0;
                while (!produced) {
                    #pragma omp critical
                    {
                        if (count < BUFFER_SIZE) {
                            buffer[in] = i;
                            in = (in + 1) % BUFFER_SIZE;
                            count++;
                            produce_item(i);
                            produced = 1;
                        }
                    }
                }
            }
        }
        else if (thread_id == 1) {
            for (i = 1; i <= NUM_ITEMS; i++) {
                int consumed = 0;
                while (!consumed) {
                    #pragma omp critical
                    {
                        if (count > 0) {
                            int item = buffer[out];
                            out = (out + 1) % BUFFER_SIZE;
                            count--;
                            consume_item(item);
                            consumed = 1;
                        }
                    }
                }
            }
        }
    }
}

```

```

    }
    }
    }
    }
    }

    return 0;
}

```

Matrix Multiplication

```

#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

#define MAX_THREADS 16
#define MAX_SIZES 3

int main() {
    int i, j, k, t, size_idx;
    double **A, **B, **C;
    double start_time, end_time;

    int sizes[MAX_SIZES] = {200, 400, 600};

    FILE *fp = fopen("PS_1.csv", "a");
    if (fp == NULL) {
        printf("Error opening PS_1.csv!\n");
        return 1;
    }

    fseek(fp, 0, SEEK_END);
    if (ftell(fp) == 0) {
        fprintf(fp, "MatrixSize,Threads,ExecutionTime\n");
    }

    printf("%-12s %-7s %s\n", "DataSize", "Threads", "Time(s)");
    printf("-----\n");

    for (size_idx = 0; size_idx < MAX_SIZES; size_idx++) {
        int N = sizes[size_idx];

        A = (double **)malloc(N * sizeof(double *));
        B = (double **)malloc(N * sizeof(double *));
        C = (double **)malloc(N * sizeof(double *));
    }
}

```

```

for (i = 0; i < N; i++) {
    A[i] = (double *)malloc(N * sizeof(double));
    B[i] = (double *)malloc(N * sizeof(double));
    C[i] = (double *)malloc(N * sizeof(double));
}

for (i = 0; i < N; i++) {
    for (j = 0; j < N; j++) {
        A[i][j] = rand() % 100;
        B[i][j] = rand() % 100;
        C[i][j] = 0.0;
    }
}

for (t = 1; t <= MAX_THREADS; t *= 2) {
    omp_set_num_threads(t);

    start_time = omp_get_wtime();

    #pragma omp parallel for private(i, j, k) shared(A, B, C)
    for (i = 0; i < N; i++) {
        for (j = 0; j < N; j++) {
            double sum = 0.0;
            for (k = 0; k < N; k++) {
                sum += A[i][k] * B[k][j];
            }
            C[i][j] = sum;
        }
    }

    end_time = omp_get_wtime();

    double exec_time = end_time - start_time;
    printf("%-12d %-7d %.6f\n", N, t, exec_time);

    fprintf(fp, "%d,%d,%.6f\n", N, t, exec_time);
}

for (i = 0; i < N; i++) {
    free(A[i]);
    free(B[i]);
    free(C[i]);
}
free(A);
free(B);
free(C);
}

fclose(fp);

```

```
    return 0;
}
```

Matrix Scalar Multiplication

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

int main() {
    int sizes[] = {100, 200, 300, 400, 500};
    int num_sizes = sizeof(sizes) / sizeof(sizes[0]);
    int i, j, t, s, N, M;
    double **A;
    double scalar = 5.0;
    double start_time, end_time, exec_time;
    FILE *fptr;

    fptr = fopen("PS_2.csv", "w");
    if (fptr == NULL) {
        printf("Error opening file!\n");
        return 1;
    }
    fprintf(fptr, "DataSize,Threads,Time(s)\n");

    printf("%-12s %-7s %s\n", "DataSize", "Threads", "Time(s)");
    printf("-----\n");

    for (s = 0; s < num_sizes; s++) {
        N = sizes[s];
        M = sizes[s];

        A = (double **)malloc(N * sizeof(double *));
        for (i = 0; i < N; i++) {
            A[i] = (double *)malloc(M * sizeof(double));
        }

        for (i = 0; i < N; i++) {
            for (j = 0; j < M; j++) {
                A[i][j] = rand() % 100;
            }
        }

        for (t = 1; t <= 8; t *= 2) {
            omp_set_num_threads(t);
```

```

        start_time = omp_get_wtime();

        #pragma omp parallel for private(i, j) shared(A, scalar, N, M)
        for (i = 0; i < N; i++) {
            for (j = 0; j < M; j++) {
                A[i][j] = A[i][j] * scalar;
            }
        }

        end_time = omp_get_wtime();
        exec_time = end_time - start_time;

        printf("%-12d %-7d %f\n", N*M, t, exec_time);
        fprintf(fp, "%d,%d,%f\n", N*M, t, exec_time);
    }

    for (i = 0; i < N; i++) {
        free(A[i]);
    }
    free(A);
}

fclose(fp);
return 0;
}

```

Matrix Vector Multiplication

```

#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

void run_matrix_vector_mult(int N, int M, int threads, FILE *fp) {
    int i, j;
    double **A, *x, *y;
    double start_time, end_time;

    A = (double **)malloc(N * sizeof(double *));
    for (i = 0; i < N; i++) {
        A[i] = (double *)malloc(M * sizeof(double));
    }
    x = (double *)malloc(M * sizeof(double));
    y = (double *)malloc(N * sizeof(double));

    for (i = 0; i < N; i++) {
        for (j = 0; j < M; j++) {

```



```

        A[i][j] = rand() % 100;
    }
}
for (j = 0; j < M; j++) {
    x[j] = rand() % 100;
}

omp_set_num_threads(threads);
start_time = omp_get_wtime();

#pragma omp parallel for private(i, j) shared(A, x, y)
for (i = 0; i < N; i++) {
    double sum = 0.0;
    for (j = 0; j < M; j++) {
        sum += A[i][j] * x[j];
    }
    y[i] = sum;
}

end_time = omp_get_wtime();
double exec_time = end_time - start_time;

printf("%-12d %-7d %f\n", N*M, threads, exec_time);

fprintf(fp, "%d,%d,%f\n", N*M, threads, exec_time);

for (i = 0; i < N; i++) free(A[i]);
free(A);
free(x);
free(y);
}

int main() {
    int sizes[] = {200, 500, 1000, 1500};
    int num_sizes = sizeof(sizes) / sizeof(sizes[0]);
    int thread_counts[] = {1, 2, 4, 8};
    int num_threads = sizeof(thread_counts) / sizeof(thread_counts[0]);

    FILE *fp = fopen("PS_3.csv", "w");
    if (!fp) {
        printf("Error opening file!\n");
        return 1;
    }

    // CSV Header
    fprintf(fp, "DataSize,Threads,Time(s)\n");

    // Print Header
    printf("%-12s %-7s %s\n", "DataSize", "Threads", "Time(s)");
    printf("-----\n");

```

```

// Run for different sizes and thread counts
for (int i = 0; i < num_sizes; i++) {
    for (int j = 0; j < num_threads; j++) {
        run_matrix_vector_mult(sizes[i], sizes[i], thread_counts[j],
fp);
    }
}

fclose(fp);
return 0;
}

```

Prefix Sum

```

#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

void run_prefix_sum(int N, int num_threads, FILE *csv_file) {
    int i;
    double *arr, *prefix;
    double start_time, end_time;

    arr = (double *)malloc(N * sizeof(double));
    prefix = (double *)malloc(N * sizeof(double));

    for (i = 0; i < N; i++) {
        arr[i] = 1;
    }

    omp_set_num_threads(num_threads);

    start_time = omp_get_wtime();

    #pragma omp parallel
    {
        int id = omp_get_thread_num();
        int nthreads = omp_get_num_threads();
        int chunk_size = N / nthreads;
        int start = id * chunk_size;
        int end = (id == nthreads - 1) ? N : start + chunk_size;

        prefix[start] = arr[start];
        for (i = start + 1; i < end; i++) {
            prefix[i] = prefix[i - 1] + arr[i];

```

```

    }

    double block_sum = prefix[end - 1];

    #pragma omp barrier
    double offset = 0.0;
    for (i = 0; i < id; i++) {
        int prev_end = (i + 1) * chunk_size;
        offset += prefix[prev_end - 1];
    }

    if (id != 0) {
        for (i = start; i < end; i++) {
            prefix[i] += offset;
        }
    }
}

end_time = omp_get_wtime();

double exec_time = end_time - start_time;

printf("%-12d %-7d %f\n", N, num_threads, exec_time);

fprintf(csv_file, "%d,%d,%f\n", N, num_threads, exec_time);

free(arr);
free(prefix);
}

int main() {
    int data_sizes[] = {10000, 50000, 100000, 500000, 1000000};
    int num_sizes = sizeof(data_sizes) / sizeof(data_sizes[0]);

    int thread_counts[] = {1, 2, 4, 8};
    int num_threads = sizeof(thread_counts) / sizeof(thread_counts[0]);

    FILE *csv_file = fopen("PS_4.csv", "w");
    if (!csv_file) {
        printf("Error opening CSV file!\n");
        return 1;
    }

    fprintf(csv_file, "DataSize,Threads,Time(s)\n");

    printf("%-12s %-7s %s\n", "DataSize", "Threads", "Time(s)");
    printf("-----\n");

    for (int i = 0; i < num_sizes; i++) {
        for (int j = 0; j < num_threads; j++) {

```

```

        run_prefix_sum(data_sizes[i], thread_counts[j], csv_file);
    }
}

fclose(csv_file);
return 0;
}

```

Sum of Squares

```

#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

int main() {
    long long N;
    printf("Enter N: ");
    scanf("%lld", &N);

    double start, end;
    long long i;
    double sum_no_reduction = 0.0;
    double sum_with_reduction = 0.0;

    // ----- Without Reduction -----
    start = omp_get_wtime();

    #pragma omp parallel for private(i) shared(sum_no_reduction)
    for (i = 1; i <= N; i++) {
        double local = i * i;
        #pragma omp critical
        sum_no_reduction += local;
    }

    end = omp_get_wtime();
    double time_no_reduction = end - start;

    // ----- With Reduction -----
    start = omp_get_wtime();

    #pragma omp parallel for reduction(+:sum_with_reduction)
    for (i = 1; i <= N; i++) {
        sum_with_reduction += i * i;
    }

    end = omp_get_wtime();
}

```

```

double time_with_reduction = end - start;

// ----- Results -----
printf("\nResults for N = %lld\n", N);
printf("Sum (no reduction): %.0f\n", sum_no_reduction);
printf("Sum (with reduction): %.0f\n", sum_with_reduction);
printf("\nExecution Time:\n");
printf("Without reduction : %.6f s\n", time_no_reduction);
printf("With reduction      : %.6f s\n", time_with_reduction);

return 0;
}

```

Prefix Sum

Naive Parallelization

```

#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

int main() {
    int N = 1 << 20; // 1 million
    int *A = malloc(N * sizeof(int));
    int *P = malloc(N * sizeof(int));
    int num_threads;

    for (int i = 0; i < N; i++) A[i] = 1;

    double start = omp_get_wtime();

    #pragma omp parallel
    {
        int tid = omp_get_thread_num();
        int nthreads = omp_get_num_threads();
        num_threads = nthreads;

        int chunk = N / nthreads;
        int start_i = tid * chunk;
        int end_i = (tid == nthreads - 1) ? N : start_i + chunk;

        // Step 1: Local prefix sum
        P[start_i] = A[start_i];
        for (int i = start_i + 1; i < end_i; i++) {
            P[i] = P[i - 1] + A[i];
        }
    }
}

```

```

        // Step 2: Compute total sum of each chunk (per thread)
        int local_sum = P[end_i - 1];

        // Step 3: Synchronize partial sums
        #pragma omp barrier

        // Step 4: Compute offsets and adjust results
        int offset = 0;
        for (int i = 0; i < tid; i++) {
            offset += P[(i + 1) * chunk - 1];
        }

        for (int i = start_i; i < end_i; i++) {
            P[i] += offset;
        }
    }

    double end = omp_get_wtime();
    printf("Naive Parallel Prefix Sum done in %.6f s with %d threads\n",
           end - start, num_threads);

    free(A);
    free(P);
    return 0;
}

```

Work Efficient

```

#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

void parallel_prefix_sum(int *A, int *P, int N) {
    int d;

    // Up-sweep (reduce) phase
    for (d = 1; d < N; d *= 2) {
        #pragma omp parallel for
        for (int i = d * 2 - 1; i < N; i += d * 2) {
            A[i] += A[i - d];
        }
    }

    // Clear last element
    A[N - 1] = 0;

    // Down-sweep phase
    for (d = N / 2; d >= 1; d /= 2) {

```

```

        #pragma omp parallel for
        for (int i = d * 2 - 1; i < N; i += d * 2) {
            int t = A[i - d];
            A[i - d] = A[i];
            A[i] += t;
        }
    }

    // Copy to output
    #pragma omp parallel for
    for (int i = 0; i < N; i++)
        P[i] = A[i];
}

int main() {
    int N = 1 << 20; // must be a power of 2
    int *A = malloc(N * sizeof(int));
    int *P = malloc(N * sizeof(int));

    for (int i = 0; i < N; i++) A[i] = 1;

    double start = omp_get_wtime();
    parallel_prefix_sum(A, P, N);
    double end = omp_get_wtime();

    printf("Work-Efficient Prefix Sum done in %.6f s\n", end - start);
    free(A);
    free(P);
    return 0;
}

```

Matrix Transpose

```

#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

int main() {
    int N = 2000;
    double **A, **B;

    A = malloc(N * sizeof(double *));
    B = malloc(N * sizeof(double *));
    for (int i = 0; i < N; i++) {
        A[i] = malloc(N * sizeof(double));
        B[i] = malloc(N * sizeof(double));
    }
}

```

```

        for (int j = 0; j < N; j++)
            A[i][j] = i * N + j; // initialize with some values
    }

    double start = omp_get_wtime();

    #pragma omp parallel for collapse(2) schedule(static)
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            B[j][i] = A[i][j];
        }
    }

    double end = omp_get_wtime();
    printf("Naive transpose time: %.6f s\n", end - start);

    for (int i = 0; i < N; i++) {
        free(A[i]);
        free(B[i]);
    }
    free(A);
    free(B);
    return 0;
}

```

Prime Count

```

#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

int is_prime(long n) {
    if (n < 2) return 0;
    if (n == 2 || n == 3) return 1;
    if (n % 2 == 0 || n % 3 == 0) return 0;
    for (long i = 5; i * i <= n; i += 6)
        if (n % i == 0 || n % (i + 2) == 0)
            return 0;
    return 1;
}

int main() {
    long N = 1000000; // 1 million
    int num_threads = 8;
    omp_set_num_threads(num_threads);
}

```



```

double start, end;
long count_static = 0, count_dynamic = 0;

// --- Static Scheduling ---
start = omp_get_wtime();

#pragma omp parallel for schedule(static) reduction(+:count_static)
for (long i = 2; i <= N; i++) {
    if (is_prime(i))
        count_static++;
}

end = omp_get_wtime();
double time_static = end - start;
printf("Static schedule:  %ld primes found in %.6f s\n", count_static,
time_static);

// --- Dynamic Scheduling ---
start = omp_get_wtime();

#pragma omp parallel for schedule(dynamic, 100)
reduction(+:count_dynamic)
for (long i = 2; i <= N; i++) {
    if (is_prime(i))
        count_dynamic++;
}

end = omp_get_wtime();
double time_dynamic = end - start;
printf("Dynamic schedule:  %ld primes found in %.6f s\n", count_dynamic,
time_dynamic);

printf("\nSpeedup ratio (static/dynamic): %.3f\n", time_static /
time_dynamic);

return 0;
}

```

Parallel Histogram

```

#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

#define N 1000000 // Size of input array
#define MAX_VALUE 10 // Values range from 0 to MAX_VALUE-1

```

```

int main() {
    int *arr = (int*) malloc(N * sizeof(int));
    int *hist = (int*) calloc(MAX_VALUE, sizeof(int));
    int i, method;
    double start, end;

    // Initialize array with random numbers between 0 and MAX_VALUE-1
    for (i = 0; i < N; i++)
        arr[i] = rand() % MAX_VALUE;

    printf("Choose method: 1-critical 2-atomic 3-reduction\n");
    scanf("%d", &method);

    start = omp_get_wtime();

    if (method == 1) {
        // ----- Method 1: Using critical section -----
        #pragma omp parallel for
        for (i = 0; i < N; i++) {
            int value = arr[i];
            #pragma omp critical
            {
                hist[value]++;
            }
        }
    }
    else if (method == 2) {
        // ----- Method 2: Using atomic operation -----
        #pragma omp parallel for
        for (i = 0; i < N; i++) {
            int value = arr[i];
            #pragma omp atomic
            hist[value]++;
        }
    }
    else if (method == 3) {
        // ----- Method 3: Using reduction (manual technique) -----
        int local_hist[MAX_VALUE] = {0};
        #pragma omp parallel private(local_hist)
        {
            // Each thread creates a local histogram
            for (int i = 0; i < MAX_VALUE; i++)
                local_hist[i] = 0;

            #pragma omp for nowait
            for (int i = 0; i < N; i++) {
                local_hist[arr[i]]++;
            }
        }
    }
}

```

```

        // Combine local histograms into global one
        #pragma omp critical
        {
            for (int i = 0; i < MAX_VALUE; i++)
                hist[i] += local_hist[i];
        }
    }
}

end = omp_get_wtime();

printf("\nHistogram results:\n");
for (i = 0; i < MAX_VALUE; i++)
    printf("Value %d: %d\n", i, hist[i]);

printf("\nExecution Time: %f seconds\n", end - start);

free(arr);
free(hist);
return 0;
}

```

Bank Transcription Simulation

```

#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

#define NUM_THREADS 4
#define NUM_TRANSACTIONS 1000000

int main() {
    double balance = 1000.0; // initial balance
    double deposit_amount = 5.0;
    double withdraw_amount = 5.0;
    int choice;
    double start, end;

    printf("Choose method: 1-critical 2-locks\n");
    scanf("%d", &choice);

    omp_set_num_threads(NUM_THREADS);
    start = omp_get_wtime();

    if (choice == 1) {
        // ----- METHOD 1: Using critical -----
    }
}

```

```

#pragma omp parallel
{
    int tid = omp_get_thread_num();
    for (int i = 0; i < NUM_TRANSACTIONS; i++) {
        #pragma omp critical
        {
            if (i % 2 == 0)
                balance += deposit_amount; // deposit
            else
                balance -= withdraw_amount; // withdrawal
        }
    }
}

} else if (choice == 2) {
    // ----- METHOD 2: Using locks -----
    omp_lock_t lock;
    omp_init_lock(&lock);

    #pragma omp parallel
    {
        int tid = omp_get_thread_num();
        for (int i = 0; i < NUM_TRANSACTIONS; i++) {
            omp_set_lock(&lock);
            if (i % 2 == 0)
                balance += deposit_amount;
            else
                balance -= withdraw_amount;
            omp_unset_lock(&lock);
        }
    }

    omp_destroy_lock(&lock);
}

end = omp_get_wtime();

printf("\nFinal balance = %.2f\n", balance);
printf("Execution time = %f seconds\n", end - start);

return 0;
}

```

Ordered Printing

```

#include <stdio.h>
#include <omp.h>

int main() {
    int N = 20;
    int i;

    printf("Parallel computation, ordered printing:\n\n");

    #pragma omp parallel for ordered schedule(static)
    for (i = 1; i <= N; i++) {
        int square = i * i; // some computation

        // Ensure print order matches iteration order
        #pragma omp ordered
        {
            printf("Thread %d → i = %2d, square = %3d\n",
omp_get_thread_num(), i, square);
        }
    }

    return 0;
}

```

Sparse Matrix Multiplication

```

#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

#define N 1000 // Matrix dimension
#define SPARSITY 0.90 // 90% zeros (i.e., only 10% nonzero)
#define NUM_THREADS 8

// CSR data structure
typedef struct {
    int *row_ptr;
    int *col_ind;
    double *values;
    int nrows, ncols, nnz;
} CSRMatrix;

// Generate random sparse matrix in CSR format
CSRMatrix *generate_sparse_matrix(int n, double sparsity) {
    CSRMatrix *mat = (CSRMatrix *)malloc(sizeof(CSRMatrix));
    mat->nrows = mat->ncols = n;

```

```

int max_nnz = n * n * (1.0 - sparsity);
mat->values = (double *)malloc(max_nnz * sizeof(double));
mat->col_ind = (int *)malloc(max_nnz * sizeof(int));
mat->row_ptr = (int *)malloc((n + 1) * sizeof(int));

int nnz = 0;
mat->row_ptr[0] = 0;
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        double r = (double)rand() / RAND_MAX;
        if (r > sparsity) {
            mat->values[nnz] = rand() % 10 + 1;
            mat->col_ind[nnz] = j;
            nnz++;
        }
    }
    mat->row_ptr[i + 1] = nnz;
}

mat->nnz = nnz;
return mat;
}

// Sparse matrix-vector multiplication (CSR)
void spmv_csr(CSRMatrix *A, double *x, double *y) {
    #pragma omp parallel for num_threads(NUM_THREADS)
    for (int i = 0; i < A->nrows; i++) {
        double sum = 0.0;
        for (int idx = A->row_ptr[i]; idx < A->row_ptr[i + 1]; idx++) {
            sum += A->values[idx] * x[A->col_ind[idx]];
        }
        y[i] = sum;
    }
}

// Dense matrix-vector multiplication
void dense_mv(double **A, double *x, double *y, int n) {
    #pragma omp parallel for num_threads(NUM_THREADS)
    for (int i = 0; i < n; i++) {
        double sum = 0.0;
        for (int j = 0; j < n; j++)
            sum += A[i][j] * x[j];
        y[i] = sum;
    }
}

int main() {
    srand(42);
    omp_set_num_threads(NUM_THREADS);

```

```

// ----- Dense matrix -----
double **A = (double **)malloc(N * sizeof(double *));
for (int i = 0; i < N; i++) {
    A[i] = (double *)malloc(N * sizeof(double));
    for (int j = 0; j < N; j++)
        A[i][j] = (rand() % 100 < (100 * (1 - SPARSITY))) ? rand() % 10
+ 1 : 0;
}

double *x = (double *)malloc(N * sizeof(double));
double *y_dense = (double *)malloc(N * sizeof(double));
double *y_sparse = (double *)malloc(N * sizeof(double));
for (int i = 0; i < N; i++) x[i] = 1.0;

// ----- Sparse version -----
CSRMatrix *A_sparse = generate_sparse_matrix(N, SPARSITY);

double start, end;

// --- Dense MV ---
start = omp_get_wtime();
dense_mv(A, x, y_dense, N);
end = omp_get_wtime();
printf("Dense MV Time: %.6f s\n", end - start);

// --- Sparse MV (CSR) ---
start = omp_get_wtime();
spmv_csr(A_sparse, x, y_sparse);
end = omp_get_wtime();
printf("Sparse MV Time: %.6f s\n", end - start);

// --- Sanity check ---
printf("\nSample results (first 5 elements):\n");
for (int i = 0; i < 5; i++)
    printf("y_dense[%d] = %.2f, y_sparse[%d] = %.2f\n",
        i, y_dense[i], i, y_sparse[i]);

// Cleanup
for (int i = 0; i < N; i++) free(A[i]);
free(A);
free(x);
free(y_dense);
free(y_sparse);
free(A_sparse->values);
free(A_sparse->col_ind);
free(A_sparse->row_ptr);
free(A_sparse);

```

```
    return 0;
}
```

Odd Even Sort

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

#define NUM_THREADS 8

// Function to perform Odd-Even Sort
void odd_even_sort(int *arr, int n) {
    int phase, i, temp;

    for (phase = 0; phase < n; phase++) {
        if (phase % 2 == 0) {
            // Even phase: (0,1), (2,3), ...
            #pragma omp parallel for num_threads(NUM_THREADS) private(i,
temp) shared(arr)
            for (i = 0; i < n - 1; i += 2) {
                if (arr[i] > arr[i + 1]) {
                    temp = arr[i];
                    arr[i] = arr[i + 1];
                    arr[i + 1] = temp;
                }
            }
        } else {
            // Odd phase: (1,2), (3,4), ...
            #pragma omp parallel for num_threads(NUM_THREADS) private(i,
temp) shared(arr)
            for (i = 1; i < n - 1; i += 2) {
                if (arr[i] > arr[i + 1]) {
                    temp = arr[i];
                    arr[i] = arr[i + 1];
                    arr[i + 1] = temp;
                }
            }
        }
    }
}

int main() {
    int N;
    printf("Enter array size: ");
    scanf("%d", &N);
```



```

int *arr = (int *)malloc(N * sizeof(int));
for (int i = 0; i < N; i++)
    arr[i] = rand() % 10000;

double start = omp_get_wtime();
odd_even_sort(arr, N);
double end = omp_get_wtime();

printf("\nSorted Array (first 10 elements): ");
for (int i = 0; i < (N < 10 ? N : 10); i++)
    printf("%d ", arr[i]);
printf("\nExecution Time: %.6f seconds\n", end - start);

free(arr);
return 0;
}

```

Jacobi

```

#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

#define N 1000          // Grid size
#define ITER 500        // Number of iterations

int main() {
    static double A[N][N], Anew[N][N];
    double start, end;

    // Initialize grid
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            A[i][j] = (i == 0 || j == 0 || i == N-1 || j == N-1) ? 1.0 :
0.0;

    // =====
    // 1. Outer loop parallelization
    // =====
    start = omp_get_wtime();
    for (int iter = 0; iter < ITER; iter++) {
        #pragma omp parallel for
        for (int i = 1; i < N - 1; i++) {
            for (int j = 1; j < N - 1; j++) {
                Anew[i][j] = 0.25 * (A[i-1][j] + A[i+1][j] + A[i][j-1] +

```

```

A[i][j+1]);
    }
}

// Copy back
#pragma omp parallel for
for (int i = 1; i < N - 1; i++)
    for (int j = 1; j < N - 1; j++)
        A[i][j] = Anew[i][j];
}
end = omp_get_wtime();
printf("Outer loop parallel time: %f seconds\n", end - start);

// =====
// 2. Collapsed loop parallelization
// =====
// Reinitialize grid
for (int i = 0; i < N; i++)
    for (int j = 0; j < N; j++)
        A[i][j] = (i == 0 || j == 0 || i == N-1 || j == N-1) ? 1.0 :
0.0;

start = omp_get_wtime();
for (int iter = 0; iter < ITER; iter++) {
    #pragma omp parallel for collapse(2)
    for (int i = 1; i < N - 1; i++) {
        for (int j = 1; j < N - 1; j++) {
            Anew[i][j] = 0.25 * (A[i-1][j] + A[i+1][j] + A[i][j-1] +
A[i][j+1]);
        }
    }

    #pragma omp parallel for collapse(2)
    for (int i = 1; i < N - 1; i++)
        for (int j = 1; j < N - 1; j++)
            A[i][j] = Anew[i][j];
}
end = omp_get_wtime();
printf("Collapsed loop parallel time: %f seconds\n", end - start);

return 0;
}

```

Monte Carlo Estimation

```

#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
#include <math.h>

int main() {
    long long int total_points = 100000000; // 1e8 points
    long long int inside_circle = 0;
    double x, y, pi;
    double start, end;

    srand(42); // for reproducibility

    start = omp_get_wtime();

    #pragma omp parallel for private(x, y) reduction(+:inside_circle)
    for (long long int i = 0; i < total_points; i++) {
        // Each thread should use its own RNG to avoid contention
        unsigned int seed = omp_get_thread_num() + i;
        x = (double)rand_r(&seed) / RAND_MAX;
        y = (double)rand_r(&seed) / RAND_MAX;

        if (x * x + y * y <= 1.0)
            inside_circle++;
    }

    end = omp_get_wtime();

    pi = 4.0 * (double)inside_circle / (double)total_points;

    printf("Estimated  $\pi$  = %.10f\n", pi);
    printf("Execution time = %.4f seconds\n", end - start);

    return 0;
}

```

Word File Count

```

#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>
#include <omp.h>

// Helper: count words in a text buffer
long count_words_in_chunk(char *buffer, long start, long end) {
    long count = 0;

```

```

    int in_word = 0;

    for (long i = start; i < end; i++) {
        if (isalnum(buffer[i])) {
            if (!in_word) {
                in_word = 1;
                count++;
            }
        } else {
            in_word = 0;
        }
    }

    return count;
}

int main(int argc, char *argv[]) {
    if (argc < 2) {
        printf("Usage: %s <filename>\n", argv[0]);
        return 1;
    }

    FILE *fp = fopen(argv[1], "rb");
    if (!fp) {
        perror("File open failed");
        return 1;
    }

    // Get file size
    fseek(fp, 0, SEEK_END);
    long filesize = ftell(fp);
    rewind(fp);

    // Load file into memory
    char *buffer = (char *)malloc(filesize);
    if (!buffer) {
        printf("Memory allocation failed\n");
        fclose(fp);
        return 1;
    }

    fread(buffer, 1, filesize, fp);
    fclose(fp);

    long total_words = 0;
    double start = omp_get_wtime();

    #pragma omp parallel
    {
        int tid = omp_get_thread_num();

```

```

    int num_threads = omp_get_num_threads();

    long chunk_size = filesize / num_threads;
    long start_idx = tid * chunk_size;
    long end_idx = (tid == num_threads - 1) ? filesize : start_idx +
chunk_size;

    // Adjust start to word boundary
    if (start_idx > 0) {
        while (start_idx < filesize && isalnum(buffer[start_idx])) {
            start_idx++;
        }
    }

    long local_count = count_words_in_chunk(buffer, start_idx, end_idx);

    #pragma omp atomic
    total_words += local_count;
}

double end = omp_get_wtime();

printf("Total words: %ld\n", total_words);
printf("Execution time: %.6f seconds\n", end - start);

free(buffer);
return 0;
}

```

BFS

```

#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

#define MAX_NODES 10000

typedef struct {
    int *neighbors;
    int degree;
} Node;

void bfs(Node *graph, int N, int start) {
    int *visited = (int *)calloc(N, sizeof(int));
    int *frontier = (int *)malloc(N * sizeof(int));
    int *next_frontier = (int *)malloc(N * sizeof(int));
}

```

```

int frontier_size = 1;
frontier[0] = start;
visited[start] = 1;

int level = 0;

double start_time = omp_get_wtime();

while (frontier_size > 0) {
    int next_frontier_size = 0;

    #pragma omp parallel
    {
        int *local_frontier = (int *)malloc(N * sizeof(int));
        int local_size = 0;

        #pragma omp for schedule(dynamic)
        for (int i = 0; i < frontier_size; i++) {
            int u = frontier[i];

            for (int j = 0; j < graph[u].degree; j++) {
                int v = graph[u].neighbors[j];
                if (!visited[v]) {
                    int was_visited = 0;

                    #pragma omp critical
                    {
                        if (!visited[v]) {
                            visited[v] = 1;
                        } else {
                            was_visited = 1;
                        }
                    }

                    if (!was_visited) {
                        local_frontier[local_size++] = v;
                    }
                }
            }
        }

        #pragma omp critical
        {
            for (int k = 0; k < local_size; k++) {
                next_frontier[next_frontier_size++] = local_frontier[k];
            }
        }

        free(local_frontier);
    }
}

```

```

    }

    frontier_size = next_frontier_size;

    // swap buffers
    int *tmp = frontier;
    frontier = next_frontier;
    next_frontier = tmp;

    level++;
}

double end_time = omp_get_wtime();

printf("BFS completed in %.6f seconds (levels = %d)\n", end_time -
start_time, level);

free(visited);
free(frontier);
free(next_frontier);
}

int main() {
    int N = 1000;
    Node *graph = (Node *)malloc(N * sizeof(Node));

    // Generate a random sparse graph
    for (int i = 0; i < N; i++) {
        graph[i].degree = rand() % 10;
        graph[i].neighbors = (int *)malloc(graph[i].degree * sizeof(int));
        for (int j = 0; j < graph[i].degree; j++) {
            graph[i].neighbors[j] = rand() % N;
        }
    }

    bfs(graph, N, 0);

    for (int i = 0; i < N; i++)
        free(graph[i].neighbors);
    free(graph);

    return 0;
}

```

Mandelbrot Set

```

#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

#define WIDTH 1000
#define HEIGHT 1000
#define MAX_ITER 1000

int mandelbrot(double x0, double y0) {
    double x = 0.0, y = 0.0;
    int iter = 0;
    while (x * x + y * y <= 4.0 && iter < MAX_ITER) {
        double xtemp = x * x - y * y + x0;
        y = 2 * x * y + y0;
        x = xtemp;
        iter++;
    }
    return iter;
}

int main() {
    double xmin = -2.0, xmax = 1.0;
    double ymin = -1.5, ymax = 1.5;
    double dx = (xmax - xmin) / WIDTH;
    double dy = (ymax - ymin) / HEIGHT;

    int *image = (int *)malloc(WIDTH * HEIGHT * sizeof(int));

    double start = omp_get_wtime();

    #pragma omp parallel for schedule(dynamic)
    for (int i = 0; i < HEIGHT; i++) {
        double y0 = ymin + i * dy;
        for (int j = 0; j < WIDTH; j++) {
            double x0 = xmin + j * dx;
            image[i * WIDTH + j] = mandelbrot(x0, y0);
        }
    }

    double end = omp_get_wtime();

    printf("Mandelbrot generated in %.3f seconds\n", end - start);

    // Optional: Save as PGM (grayscale image)
    FILE *fp = fopen("mandelbrot.pgm", "w");
    fprintf(fp, "P2\n%d %d\n%d\n", WIDTH, HEIGHT, MAX_ITER);
    for (int i = 0; i < HEIGHT * WIDTH; i++) {
        fprintf(fp, "%d ", image[i]);
    }
}

```



```
    fclose(fp);

    free(image);
    return 0;
}
```

Maze Solver

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

#define N 8
#define M 8

// 0 = path, 1 = wall
int maze[N][M] = {
    {0,0,1,0,0,0,1,0},
    {1,0,1,0,1,0,0,0},
    {0,0,0,0,1,0,1,0},
    {0,1,1,1,1,0,1,0},
    {0,0,0,0,0,0,1,0},
    {1,1,1,1,1,0,1,0},
    {0,0,0,0,1,0,0,0},
    {0,1,1,0,0,0,1,0}
};

int visited[N][M];
int found_exit = 0;

// Directions: up, down, left, right
int dirs[4][2] = {{-1,0},{1,0},{0,-1},{0,1}};

typedef struct {
    int x, y;
} Cell;

int is_valid(int x, int y) {
    return (x >= 0 && y >= 0 && x < N && y < M && maze[x][y] == 0 &&
!visited[x][y]);
}

void parallel_bfs(int start_x, int start_y, int exit_x, int exit_y) {
    Cell *frontier = malloc(N * M * sizeof(Cell));
    Cell *next_frontier = malloc(N * M * sizeof(Cell));
    int frontier_size = 1, next_size = 0;
```

```

frontier[0].x = start_x;
frontier[0].y = start_y;
visited[start_x][start_y] = 1;

int level = 0;

while (frontier_size > 0 && !found_exit) {
    next_size = 0;

    #pragma omp parallel for shared(frontier, next_frontier, found_exit)
    reduction(+:next_size)
    for (int i = 0; i < frontier_size; i++) {
        int x = frontier[i].x;
        int y = frontier[i].y;

        if (x == exit_x && y == exit_y) {
            found_exit = 1;
            continue;
        }

        for (int d = 0; d < 4; d++) {
            int nx = x + dirs[d][0];
            int ny = y + dirs[d][1];

            if (is_valid(nx, ny)) {
                #pragma omp critical
                {
                    visited[nx][ny] = 1;
                    next_frontier[next_size].x = nx;
                    next_frontier[next_size].y = ny;
                    next_size++;
                }
            }
        }
    }

    // Move to next level
    Cell *temp = frontier;
    frontier = next_frontier;
    next_frontier = temp;
    frontier_size = next_size;
    level++;
}

if (found_exit)
    printf("Exit found at level %d!\n", level);
else
    printf("No exit found.\n");

```

```

    free(frontier);
    free(next_frontier);
}

int main() {
    double start = omp_get_wtime();

    parallel_bfs(0, 0, N - 1, M - 1);

    double end = omp_get_wtime();
    printf("Time: %.6f seconds\n", end - start);

    return 0;
}

```

Sudoku Solver

```

#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

#define N 9

int is_valid(int grid[N][N], int row, int col, int num) {
    for (int x = 0; x < N; x++) {
        if (grid[row][x] == num || grid[x][col] == num)
            return 0;
    }

    int startRow = row - row % 3, startCol = col - col % 3;
    for (int i = 0; i < 3; i++)
        for (int j = 0; j < 3; j++)
            if (grid[i + startRow][j + startCol] == num)
                return 0;

    return 1;
}

int find_unassigned(int grid[N][N], int *row, int *col) {
    for (*row = 0; *row < N; (*row)++)
        for (*col = 0; *col < N; (*col)++)
            if (grid[*row][*col] == 0)
                return 1;

    return 0;
}

```

```

int solve_sudoku_seq(int grid[N][N]) {
    int row, col;
    if (!find_unassigned(grid, &row, &col))
        return 1;

    for (int num = 1; num <= 9; num++) {
        if (is_valid(grid, row, col, num)) {
            grid[row][col] = num;
            if (solve_sudoku_seq(grid))
                return 1;
            grid[row][col] = 0;
        }
    }
    return 0;
}

int solve_sudoku_parallel(int grid[N][N], int depth) {
    int row, col;
    if (!find_unassigned(grid, &row, &col))
        return 1;

    int solved = 0;

    #pragma omp parallel for shared(solved) schedule(dynamic)
    for (int num = 1; num <= 9; num++) {
        if (solved) continue; // early exit if already solved
        if (is_valid(grid, row, col, num)) {
            int local_grid[N][N];
            // Copy grid to local thread copy
            for (int i = 0; i < N; i++)
                for (int j = 0; j < N; j++)
                    local_grid[i][j] = grid[i][j];

            local_grid[row][col] = num;

            int found;
            if (depth < 1) // parallelize only top levels
                found = solve_sudoku_parallel(local_grid, depth + 1);
            else
                found = solve_sudoku_seq(local_grid);

            if (found) {
                #pragma omp critical
                {
                    if (!solved) {
                        solved = 1;
                        // Copy back solution
                        for (int i = 0; i < N; i++)
                            for (int j = 0; j < N; j++)
                                grid[i][j] = local_grid[i][j];
                    }
                }
            }
        }
    }
}

```

```

    }
    }
}

return solved;
}

void print_grid(int grid[N][N]) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++)
            printf("%d ", grid[i][j]);
        printf("\n");
    }
}

int main() {
    int grid[N][N] = {
        {5,3,0,0,7,0,0,0,0},
        {6,0,0,1,9,5,0,0,0},
        {0,9,8,0,0,0,0,6,0},
        {8,0,0,0,6,0,0,0,3},
        {4,0,0,8,0,3,0,0,1},
        {7,0,0,0,2,0,0,0,6},
        {0,6,0,0,0,0,2,8,0},
        {0,0,0,4,1,9,0,0,5},
        {0,0,0,0,8,0,0,7,9}
    };

    double start = omp_get_wtime();
    int success = solve_sudoku_parallel(grid, 0);
    double end = omp_get_wtime();

    if (success) {
        printf("Solved Sudoku:\n");
        print_grid(grid);
    } else {
        printf("No solution exists.\n");
    }

    printf("Time taken: %.6f seconds\n", end - start);

    return 0;
}

```

Graph Coloring

```

#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

#define MAX_VERTICES 10
#define MAX_COLORS 3

int graph[MAX_VERTICES][MAX_VERTICES];
int V; // number of vertices
int solution_found = 0;

// Check if it's safe to assign color c to vertex v
int is_safe(int v, int color[], int c) {
    for (int i = 0; i < V; i++) {
        if (graph[v][i] && color[i] == c)
            return 0;
    }
    return 1;
}

// Sequential recursive coloring (used at deeper levels)
int graph_coloring_seq(int color[], int v) {
    if (v == V)
        return 1;

    for (int c = 1; c <= MAX_COLORS; c++) {
        if (is_safe(v, color, c)) {
            color[v] = c;
            if (graph_coloring_seq(color, v + 1))
                return 1;
            color[v] = 0;
        }
    }
    return 0;
}

// Parallelized coloring (only at top levels)
int graph_coloring_parallel(int color[], int v, int depth) {
    if (v == V)
        return 1;

    int solved = 0;

    #pragma omp parallel for shared(solved) schedule(dynamic)
    for (int c = 1; c <= MAX_COLORS; c++) {
        if (solved) continue; // early stop

        if (is_safe(v, color, c)) {
            int local_color[MAX_VERTICES];

```

```

        for (int i = 0; i < V; i++)
            local_color[i] = color[i];

        local_color[v] = c;

        int found;
        if (depth < 1) // only parallelize first level or two
            found = graph_coloring_parallel(local_color, v + 1, depth +
1);
        else
            found = graph_coloring_seq(local_color, v + 1);

        if (found) {
            #pragma omp critical
            {
                if (!solved) {
                    solved = 1;
                    for (int i = 0; i < V; i++)
                        color[i] = local_color[i];
                }
            }
        }
    }
    return solved;
}

void print_solution(int color[]) {
    printf("Solution found:\n");
    for (int i = 0; i < V; i++)
        printf("Vertex %d -> Color %d\n", i, color[i]);
}

int main() {
    // Example: Simple 5-vertex graph
    V = 5;
    int edges[5][5] = {
        {0,1,1,1,0},
        {1,0,1,0,1},
        {1,1,0,1,1},
        {1,0,1,0,1},
        {0,1,1,1,0}
    };

    for (int i = 0; i < V; i++)
        for (int j = 0; j < V; j++)
            graph[i][j] = edges[i][j];

    int color[MAX_VERTICES] = {0};

```

```

double start = omp_get_wtime();
int success = graph_coloring_parallel(color, 0, 0);
double end = omp_get_wtime();

if (success)
    print_solution(color);
else
    printf("No valid coloring exists with %d colors.\n", MAX_COLORS);

printf("Time: %.6f seconds\n", end - start);

return 0;
}

```

Travelling Salesman

```

#include <stdio.h>
#include <stdlib.h>
#include <limits.h>
#include <omp.h>

#define N 5

int dist[N][N] = {
    {0, 10, 15, 20, 10},
    {10, 0, 35, 25, 20},
    {15, 35, 0, 30, 15},
    {20, 25, 30, 0, 20},
    {10, 20, 15, 20, 0}
};

int best_cost = INT_MAX;
#pragma omp threadprivate(best_cost) // optional per-thread copy for local best

void tsp(int *path, int *visited, int level, int cost) {
    if (level == N) {
        cost += dist[path[level - 1]][path[0]];
        #pragma omp critical
        {
            if (cost < best_cost) best_cost = cost;
        }
        return;
    }

    for (int i = 0; i < N; i++) {

```



```

        if (!visited[i]) {
            visited[i] = 1;
            path[level] = i;
            tsp(path, visited, level + 1, cost + dist[path[level - 1]][i]);
            visited[i] = 0;
        }
    }
}

int main() {
    int path[N];
    int visited[N] = {0};
    path[0] = 0;
    visited[0] = 1;

    double start = omp_get_wtime();

    #pragma omp parallel
    {
        #pragma omp single
        {
            for (int i = 1; i < N; i++) {
                #pragma omp task firstprivate(i)
                {
                    int local_path[N];
                    int local_visited[N];
                    for (int j = 0; j < N; j++) {
                        local_path[j] = path[j];
                        local_visited[j] = visited[j];
                    }
                    local_path[1] = i;
                    local_visited[i] = 1;
                    tsp(local_path, local_visited, 2, dist[0][i]);
                }
            }
        }
    }

    double end = omp_get_wtime();

    printf("Minimum TSP cost: %d\n", best_cost);
    printf("Execution time: %f seconds\n", end - start);

    return 0;
}

```

Word Search Puzzle

```

#include <stdio.h>
#include <string.h>
#include <omp.h>

#define ROWS 6
#define COLS 6
#define NUM_WORDS 4

char grid[ROWS][COLS] = {
    {'H', 'E', 'L', 'L', 'O', 'X'},
    {'A', 'B', 'C', 'D', 'E', 'Y'},
    {'L', 'O', 'V', 'E', 'S', 'Z'},
    {'O', 'P', 'E', 'N', 'M', 'P'},
    {'P', 'M', 'P', 'A', 'I', 'Q'},
    {'S', 'U', 'N', 'N', 'Y', 'R'}
};

char *words[NUM_WORDS] = {"HELLO", "OPEN", "LOVE", "SUNNY"};

int dir[8][2] = {
    {0, 1}, {1, 0}, {1, 1}, {-1, 1},
    {0, -1}, {-1, 0}, {-1, -1}, {1, -1}
};

int searchWord(char *word, int row, int col) {
    int len = strlen(word);
    for (int d = 0; d < 8; d++) {
        int rd = row, cd = col, k;
        for (k = 0; k < len; k++) {
            if (rd < 0 || rd >= ROWS || cd < 0 || cd >= COLS)
                break;
            if (grid[rd][cd] != word[k])
                break;
            rd += dir[d][0];
            cd += dir[d][1];
        }
        if (k == len)
            return 1;
    }
    return 0;
}

int main() {
    double start = omp_get_wtime();

    #pragma omp parallel
    {
        int tid = omp_get_thread_num();
        int num_threads = omp_get_num_threads();
    }
}

```

```

        // Divide rows among threads
        int rows_per_thread = ROWS / num_threads;
        int start_row = tid * rows_per_thread;
        int end_row = (tid == num_threads - 1) ? ROWS : start_row +
rows_per_thread;

        for (int i = start_row; i < end_row; i++) {
            for (int j = 0; j < COLS; j++) {
                for (int w = 0; w < NUM_WORDS; w++) {
                    if (grid[i][j] == words[w][0] && searchWord(words[w], i,
j)) {
                        #pragma omp critical
                        printf("Thread %d found '%s' starting at (%d,%d)\n",
tid, words[w], i, j);
                    }
                }
            }
        }

        double end = omp_get_wtime();
        printf("Execution time: %f seconds\n", end - start);
        return 0;
}

```

Subset Sum

```

#include <stdio.h>
#include <omp.h>

#define MAX_SIZE 20

int subsetSum(int arr[], int n, int target) {
    if (target == 0)
        return 1; // Found subset
    if (n == 0 || target < 0)
        return 0; // Dead end

    int found = 0;

    // Parallelize branching using tasks
    #pragma omp task shared(found)
    {
        if (!found)
            found = subsetSum(arr, n - 1, target - arr[n - 1]); // Include

```

```

current
{

#pragma omp task shared(found)
{
    if (!found)
        found = subsetSum(arr, n - 1, target); // Exclude current
}

#pragma omp taskwait
return found;
}

int main() {
    int arr[MAX_SIZE] = {3, 34, 4, 12, 5, 2};
    int n = 6;
    int target = 9;
    int result = 0;

    double start = omp_get_wtime();

#pragma omp parallel
{
    #pragma omp single
    result = subsetSum(arr, n, target);
}

    double end = omp_get_wtime();

    if (result)
        printf("Subset with sum %d exists.\n", target);
    else
        printf("No subset found with sum %d.\n", target);

    printf("Execution time: %f seconds\n", end - start);
    return 0;
}

```

Sliding Puzzle Solver

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <omp.h>

#define N 3

```

```

#define MAX_STATES 100000

typedef struct {
    int tiles[N][N];
    int depth;
} State;

typedef struct {
    State queue[MAX_STATES];
    int front, rear;
} Queue;

omp_lock_t queue_lock;
Queue frontier;

// Goal configuration
int goal[N][N] = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 0}
};

// Initial configuration
int start[N][N] = {
    {1, 2, 3},
    {4, 0, 6},
    {7, 5, 8}
};

// Directions for moves (up, down, left, right)
int dr[4] = {-1, 1, 0, 0};
int dc[4] = {0, 0, -1, 1};

int isGoal(State s) {
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            if (s.tiles[i][j] != goal[i][j])
                return 0;
    return 1;
}

void enqueue(State s) {
    omp_set_lock(&queue_lock);
    frontier.queue[frontier.rear++] = s;
    omp_unset_lock(&queue_lock);
}

int dequeue(State *s) {
    int success = 0;
    omp_set_lock(&queue_lock);

```

```

    if (frontier.front < frontier.rear) {
        *s = frontier.queue[frontier.front++];
        success = 1;
    }
    omp_unset_lock(&queue_lock);
    return success;
}

void copyState(State *dest, State src) {
    memcpy(dest, &src, sizeof(State));
}

void printState(State s) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++)
            printf("%d ", s.tiles[i][j]);
        printf("\n");
    }
    printf("Depth: %d\n\n", s.depth);
}

void expandState(State s) {
    int br, bc;
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            if (s.tiles[i][j] == 0) { br = i; bc = j; }

    for (int d = 0; d < 4; d++) {
        int nr = br + dr[d], nc = bc + dc[d];
        if (nr >= 0 && nr < N && nc >= 0 && nc < N) {
            State ns;
            copyState(&ns, s);
            ns.tiles[br][bc] = ns.tiles[nr][nc];
            ns.tiles[nr][nc] = 0;
            ns.depth = s.depth + 1;
            enqueue(ns);
        }
    }
}

int main() {
    omp_init_lock(&queue_lock);
    frontier.front = frontier.rear = 0;

    State initial;
    memcpy(initial.tiles, start, sizeof(start));
    initial.depth = 0;
    enqueue(initial);

    int found = 0;

```

```

double start_time = omp_get_wtime();

#pragma omp parallel shared(found)
{
    State current;
    while (!found) {
        if (!dequeue(&current))
            continue;

        if (isGoal(current)) {
            #pragma omp critical
            {
                if (!found) {
                    found = 1;
                    printf("\nGoal found by thread %d at depth %d\n",
omp_get_thread_num(), current.depth);
                    printState(current);
                }
            }
        } else if (!found) {
            expandState(current);
        }
    }
}

double end_time = omp_get_wtime();
printf("Execution time: %.4f seconds\n", end_time - start_time);

omp_destroy_lock(&queue_lock);
return 0;
}

```

Quick Sort

```

#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

// Function to swap two elements
void swap(int *a, int *b) {
    int t = *a;
    *a = *b;
    *b = t;
}

// Standard partition around a pivot

```

```

int partition(int arr[], int low, int high, int pivot_index) {
    int pivot = arr[pivot_index];
    swap(&arr[pivot_index], &arr[high]); // move pivot to end
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return i + 1;
}

// Sequential QuickSort
void quicksort_seq(int arr[], int low, int high) {
    if (low < high) {
        int pivot = partition(arr, low, high, high);
        quicksort_seq(arr, low, pivot - 1);
        quicksort_seq(arr, pivot + 1, high);
    }
}

// Speculative Multi-Pivot QuickSort
void quicksort_multi(int arr[], int low, int high, int depth) {
    if (low < high) {
        int bestPivotIndex = -1;
        int bestBalance = high - low + 1; // initialize with worst balance
        int numPivots = 3; // try 3 pivots

        int *pivot_positions = (int *)malloc(numPivots * sizeof(int));

        // Try different pivots in parallel
        #pragma omp parallel for shared(pivot_positions)
        for (int i = 0; i < numPivots; i++) {
            int pivot_index = low + rand() % (high - low + 1);
            int *tempArr = (int *)malloc((high - low + 1) * sizeof(int));
            for (int j = low; j <= high; j++) tempArr[j - low] = arr[j];

            int p = partition(tempArr, 0, high - low, pivot_index - low);
            int leftSize = p;
            int rightSize = (high - low) - p;
            int balance = abs(leftSize - rightSize);

            #pragma omp critical
            {
                if (balance < bestBalance) {
                    bestBalance = balance;
                    bestPivotIndex = pivot_index;
                }
            }
        }
    }
}

```



```

        }
        free(tempArr);
    }

    // Now partition array with the best pivot
    int pivot = partition(arr, low, high, bestPivotIndex);

    if (depth < 4) { // limit recursion depth for parallelism
        #pragma omp parallel sections
        {
            #pragma omp section
            quicksort_multi(arr, low, pivot - 1, depth + 1);

            #pragma omp section
            quicksort_multi(arr, pivot + 1, high, depth + 1);
        }
    } else {
        quicksort_seq(arr, low, pivot - 1);
        quicksort_seq(arr, pivot + 1, high);
    }

    free(pivot_positions);
}

// Utility to print array
void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");
}

int main() {
    int n = 15;
    int arr[] = {38, 27, 43, 3, 9, 82, 10, 66, 29, 5, 41, 15, 52, 1, 19};

    printf("Original array:\n");
    printArray(arr, n);

    double start = omp_get_wtime();
    quicksort_multi(arr, 0, n - 1, 0);
    double end = omp_get_wtime();

    printf("\nSorted array:\n");
    printArray(arr, n);
    printf("\nTime taken: %f seconds\n", end - start);

    return 0;
}

```

Pathfinding

```
#include <stdio.h>
#include <stdlib.h>
#include <limits.h>
#include <stdbool.h>
#include <omp.h>
#include <math.h>

#define V 6 // number of vertices

// Min distance vertex selector
int minDistance(int dist[], bool visited[]) {
    int min = INT_MAX, min_index = -1;
    for (int v = 0; v < V; v++)
        if (!visited[v] && dist[v] <= min)
            min = dist[v], min_index = v;
    return min_index;
}

// Dijkstra's algorithm
int dijkstra(int graph[V][V], int src, int target) {
    int dist[V];
    bool visited[V];
    for (int i = 0; i < V; i++) {
        dist[i] = INT_MAX;
        visited[i] = false;
    }
    dist[src] = 0;

    for (int count = 0; count < V - 1; count++) {
        int u = minDistance(dist, visited);
        if (u == -1) break;
        visited[u] = true;
        for (int v = 0; v < V; v++) {
            if (!visited[v] && graph[u][v] && dist[u] != INT_MAX &&
                dist[u] + graph[u][v] < dist[v]) {
                dist[v] = dist[u] + graph[u][v];
            }
        }
    }
    return dist[target];
}

// Simple heuristic (Euclidean) for A*
double heuristic(int u, int target, double coords[V][2]) {
    double dx = coords[u][0] - coords[target][0];
```

```

    double dy = coords[u][1] - coords[target][1];
    return sqrt(dx*dx + dy*dy);
}

// A* algorithm
int astar(int graph[V][V], int src, int target, double coords[V][2]) {
    int g[V]; bool visited[V];
    for (int i = 0; i < V; i++) { g[i] = INT_MAX; visited[i] = false; }
    g[src] = 0;

    for (int count = 0; count < V - 1; count++) {
        // Choose vertex with minimum f = g + h
        double fmin = 1e9; int u = -1;
        for (int i = 0; i < V; i++)
            if (!visited[i] && g[i] + heuristic(i, target, coords) < fmin)
                fmin = g[i] + heuristic(i, target, coords), u = i;

        if (u == -1) break;
        if (u == target) break;
        visited[u] = true;

        for (int v = 0; v < V; v++) {
            if (graph[u][v] && !visited[v]) {
                int tentative = g[u] + graph[u][v];
                if (tentative < g[v])
                    g[v] = tentative;
            }
        }
    }
    return g[target];
}

int main() {
    // Example weighted graph
    int graph[V][V] = {
        {0, 4, 2, 0, 0, 0},
        {4, 0, 5, 10, 0, 0},
        {2, 5, 0, 3, 0, 0},
        {0, 10, 3, 0, 4, 11},
        {0, 0, 0, 4, 0, 2},
        {0, 0, 0, 11, 2, 0}
    };

    double coords[V][2] = {
        {0,0}, {2,1}, {1,3}, {4,3}, {5,1}, {7,2}
    };

    int src = 0, target = 5;
    int resultD = INT_MAX, resultA = INT_MAX;

```

```

double start = omp_get_wtime();

// Run both algorithms in parallel speculatively
#pragma omp parallel sections
{
    #pragma omp section
    {
        printf("Running Dijkstra...\n");
        resultD = dijkstra(graph, src, target);
    }
    #pragma omp section
    {
        printf("Running A*...\n");
        resultA = astar(graph, src, target, coords);
    }
}

double end = omp_get_wtime();

// Choose whichever succeeded first or produced valid result
int finalResult = (resultA != INT_MAX && resultA <= resultD) ? resultA :
resultD;

printf("\nShortest path cost (winner): %d\n", finalResult);
printf("Time taken: %f seconds\n", end - start);
return 0;
}

```

Polynomial Evaluation

```

#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <omp.h>

#define N 1000000 // polynomial degree

double direct_eval(double *a, double x, int n) {
    double sum = 0.0;
    double xp = 1.0;
    for (int i = 0; i <= n; i++) {
        sum += a[i] * xp;
        xp *= x;
    }
    return sum;
}

```

```

double horner_eval(double *a, double x, int n) {
    double result = a[n];
    for (int i = n - 1; i >= 0; i--) {
        result = a[i] + x * result;
    }
    return result;
}

int main() {
    int n = 10; // degree
    double *a = (double *)malloc((n + 1) * sizeof(double));
    for (int i = 0; i <= n; i++)
        a[i] = rand() % 10 + 1;

    double x = 1.5;
    double result_horner = 0.0, result_direct = 0.0;
    double time_horner = 0.0, time_direct = 0.0;

    double start = omp_get_wtime();

    // Speculative execution: both methods run in parallel
    #pragma omp parallel sections
    {
        #pragma omp section
        {
            double t1 = omp_get_wtime();
            result_horner = horner_eval(a, x, n);
            time_horner = omp_get_wtime() - t1;
        }

        #pragma omp section
        {
            double t2 = omp_get_wtime();
            result_direct = direct_eval(a, x, n);
            time_direct = omp_get_wtime() - t2;
        }
    }

    double total_time = omp_get_wtime() - start;

    // Compare results for accuracy
    double diff = fabs(result_horner - result_direct);
    double accepted = (diff < 1e-6) ? result_horner : result_direct;

    printf("Horner Result = %.10f (time = %.6fs)\n", result_horner,
time_horner);
    printf("Direct Result = %.10f (time = %.6fs)\n", result_direct,
time_direct);
    printf("Difference      = %.10e\n", diff);

```

```
printf("Accepted Value= %.10f\n", accepted);
printf("Total Time (speculative) = %.6fs\n", total_time);

free(a);
return 0;
}
```

Matrix Multiplication

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
#include <math.h>

// Classical O(n^3)
void classical(double **A, double **B, double **C, int N) {
    for (int i=0; i<N; i++)
        for (int j=0; j<N; j++) {
            double sum = 0;
            for (int k=0; k<N; k++)
                sum += A[i][k] * B[k][j];
            C[i][j] = sum;
        }
}

// Mock Strassen's (simulated fast version for demo)
void strassen(double **A, double **B, double **C, int N) {
    // Simulated delay (e.g., recursion overhead)
    #pragma omp parallel for
    for (int i=0; i<N; i++)
        for (int j=0; j<N; j++)
            C[i][j] = A[i][j] * B[j][i] * 0.999; // slight approximation
}

int main() {
    int N = 256;
    double **A = malloc(N*sizeof(double*));
    double **B = malloc(N*sizeof(double*));
    double **C1 = malloc(N*sizeof(double*));
    double **C2 = malloc(N*sizeof(double*));
    for (int i=0; i<N; i++) {
        A[i]=malloc(N*sizeof(double));
        B[i]=malloc(N*sizeof(double));
        C1[i]=malloc(N*sizeof(double));
        C2[i]=malloc(N*sizeof(double));
        for (int j=0; j<N; j++) {
```

```

        A[i][j]=rand()%10;
        B[i][j]=rand()%10;
    }
}

double t_class, t_stras;

#pragma omp parallel sections
{
    #pragma omp section
    {
        double t1=omp_get_wtime();
        classical(A,B,C1,N);
        t_class=omp_get_wtime()-t1;
    }

    #pragma omp section
    {
        double t2=omp_get_wtime();
        strassen(A,B,C2,N);
        t_stras=omp_get_wtime()-t2;
    }
}

double diff=0;
for(int i=0;i<N;i++)
    for(int j=0;j<N;j++)
        diff += fabs(C1[i][j]-C2[i][j]);

printf("Classical Time: %.6f | Strassen Time: %.6f | Diff: %.6f\n",
        t_class, t_stras, diff);

if (t_stras < t_class && diff/N/N < 1e-3)
    printf("Accepting Strassen's result.\n");
else
    printf("Reverting to Classical.\n");

return 0;
}

```
